Guía paso a paso: correr y desplegar la API de predicción de viviendas

MLOps 2026 · Universidad EIA · Ejecución local y despliegue en Railway o Render

Al terminar esta guía vas a tener funcionando, en tu computador y en la nube, un sistema de MLOps con tres piezas: una **API con interfaz web** que predice el valor de viviendas de California, un **cron job que revisa cada 15 minutos** que la API esté sana y otro que la **reentrena los días 1 y 15 de cada mes** con datos nuevos que suben los usuarios. Si nadie sube datos, el reentrenamiento no se ejecuta y la interfaz muestra un aviso pidiéndolos.

Cómo decide el reentrenamiento

- Los usuarios suben un CSV con viviendas **y su valor real** (columna `median_house_value`).
- El cron entrena un modelo nuevo (el **retador**) con los datos originales más los subidos.
- El retador y el modelo actual (el **campeón**) se evalúan con datos que ninguno de los dos vio. El retador reemplaza al campeón solo si su error (RMSE) es menor.
- Todo queda en el **historial** de la interfaz.

> **Haz solo una de las dos plataformas**
>
> Railway (D1) **o** Render (D2). Recomendamos Railway: tiene volúmenes y cron jobs en el plan Hobby y el reentrenamiento cabe en memoria. En Render el disco persistente y los cron jobs solo existen en planes de pago y la API necesita 2 GB de RAM.

Contenido

Parte	Qué vas a hacer	Tiempo aprox.
A. Preparación	Instalar herramientas y hacer tu copia (fork) del repositorio	15 min
B. Correr en local	Levantar la API, probar los cron jobs y el ciclo de reentrenamiento	20 min
C. Token	Crear la contraseña de administración	2 min
D1. Railway	Desplegar API + 2 cron jobs (recomendado)	20 min
D2. Render	Alternativa a Railway con el Blueprint render.yaml	20 min

Parte	Qué vas a hacer	Tiempo aprox.
E. Verificar	Lista de comprobación final	10 min
Anexos	Formato del CSV y solución de problemas	-

Parte A. Preparación

1 Instala las herramientas

Herramienta	Para qué	Cómo instalarla
Git	Descargar y subir código	git-scm.com/downloads (en macOS ya viene con Xcode CLT)
uv	Python 3.12 y dependencias exactas	macOS/Linux: `curl -LsSf https://astral.sh/uv/install.sh | sh` Windows (PowerShell): `irm https://astral.sh/uv/install.ps1 | iex`
Cuenta GitHub	Railway y Render despliegan desde GitHub	github.com/signup
Cuenta Railway o Render	Hosting en la nube	railway.com · render.com (entra con GitHub)
Docker Desktop (opcional)	Probar la misma imagen que corre en la nube	docker.com/products/docker-desktop

Comprueba la instalación abriendo una terminal nueva:

```
git --version
uv --version
```

2 Haz tu copia (fork) del repositorio

Entra a **https://github.com/sebastianzapatar/mlops2026** y pulsa **Fork** (arriba a la derecha). Así tendrás **github.com/<tu-usuario>/mlops2026**, que es el que conectarás a Railway o Render.

3 Clona tu fork en tu computador

```
git clone https://github.com/<tu-usuario>/mlops2026.git
cd mlops2026
```

4 Instala las dependencias

```
uv sync
```

uv descarga Python 3.12 si no lo tienes e instala las versiones exactas de `uv.lock`. Tarda unos minutos la primera vez.

Parte B. Correr en local

Vas a usar **dos terminales** abiertas en la carpeta del proyecto: una para la API (queda corriendo) y otra para los cron jobs y las pruebas.

5 Levanta la API (terminal 1)

macOS / Linux:

```
ADMIN_TOKEN=secreto uv run uvicorn pycaret_app:app --reload
```

Windows (PowerShell):

```
$env:ADMIN_TOKEN = "secreto"
uv run uvicorn pycaret_app:app --reload
```

Deja esta terminal abierta. Cuando veas `Campeón v1 listo` en los mensajes, el modelo está preparado.

> **¿Qué pasa la primera vez?**
>
> Si existe `models/pycaret_housing.pkl` (lo exporta el cuaderno `pycaret_experimentos.ipynb`), la API lo adopta como modelo v1. Si no existe, entrena un modelo inicial con el CSV original en unos segundos.
>
> Todo se guarda en la carpeta `runtime/`. Para empezar de cero, apaga la API (Ctrl+C) y borra esa carpeta.

6 Abre la interfaz

Entra a **http://localhost:8000**. Arriba a la derecha debe decir **API en servicio** (verde). Pulsa **Predecir** con los valores que trae el formulario: verás un valor cercano a $430.000. La documentación interactiva de la API está en **http://localhost:8000/docs**.

7 Prueba los cron jobs (terminal 2)

macOS / Linux:

```
export API_URL=http://localhost:8000 ADMIN_TOKEN=secreto
uv run python cron_job.py health
uv run python cron_job.py retrain
```

Windows (PowerShell):

```
$env:API_URL = "http://localhost:8000"
$env:ADMIN_TOKEN = "secreto"
uv run python cron_job.py health
uv run python cron_job.py retrain
```

Resultado esperado:

```
[INFO] API sana · modelo v1 · predicción de prueba $...
[WARNING] SE NECESITAN DATOS NUEVOS: No hay datos nuevos: se omitió el reentrenamiento...
```

El segundo mensaje es correcto: todavía no has subido datos. Recarga la interfaz y verás el aviso rojo **Se necesitan datos nuevos**.

8 Prueba el ciclo completo de reentrenamiento

Crea un CSV de práctica con el encabezado y las primeras 200 filas del dataset (terminal 2):

macOS / Linux:

```
head -n 201 1553768847-housing.csv > nuevos.csv
```

Windows (PowerShell):

```
Get-Content 1553768847-housing.csv -TotalCount 201 | Set-Content nuevos.csv
```

- En la interfaz, sección **Datos nuevos y reentrenamiento**: escribe el token `secreto`, elige `nuevos.csv` y pulsa **Subir datos**.
- El lote aparece como **Pendiente** y el aviso superior se pone verde.
- Ejecuta otra vez `uv run python cron_job.py retrain` (o pulsa **Reentrenar ahora**).
- Mira la tabla **Historial de entrenamientos**: RMSE actual frente a RMSE nuevo y la decisión.

> **Ojo con este ejercicio**
>
> Esas 200 filas ya están en los datos originales, así que la mejora de RMSE que veas no es real; sirve solo para practicar el flujo. En producción se suben viviendas nuevas.

9 Corre las pruebas automáticas

```
uv run pytest -v
uv run flake8 *.py tests/ --max-line-length=120
```

Deben pasar las 38 pruebas. Son las mismas que ejecuta GitHub Actions en cada push a main.

10 (Opcional) Todo con Docker Compose

Con Docker Desktop abierto, un solo comando levanta la API, los cron jobs, MLflow y Jupyter. Primero crea el archivo `.env` con el token (no se sube a GitHub):

macOS / Linux:

```
cp .env.example .env
uv run python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Windows (PowerShell):

```
Copy-Item .env.example .env
uv run python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Abre `.env` y reemplaza `cambia-este-token` por el valor que imprimió el segundo comando. Luego:

```
docker compose up -d --build
```

`--build` reconstruye las imágenes si cambió el código; sin él, Compose reutiliza las que ya existan. Si el puerto 8888 está ocupado, agrega `JUPYTER_PORT=8889` a `.env`.

Servicio	Dónde	Para qué
api	http://localhost:8000	Interfaz y API
cron	`docker compose logs -f cron`	Salud cada 15 min y reentrenamiento los días 1 y 15
mlflow	http://localhost:5050	Experimentos y registro de reentrenamientos
notebook	http://localhost:8888	Jupyter (token en `docker compose logs notebook`)

- Reentrenar sin esperar al día 1 o 15: `docker compose exec cron python cron_job.py retrain`
- Apagar: `docker compose down` (el modelo, los lotes y MLflow se conservan en volúmenes).
- La primera construcción tarda varios minutos (la imagen de Jupyter instala PyCaret).

Parte C. Crear el token de administración

El token protege **subir datos** y **reentrenar**. En local usamos `secreto`, pero en la nube la API es pública en Internet: necesitas un valor largo y aleatorio.

11 Genera el token y guárdalo

```
uv run python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Cópialo en un lugar seguro (un gestor de contraseñas). Lo pegarás en la API y en el cron de reentrenamiento. **No lo subas a GitHub** ni lo pongas en el código.

Parte D1. Desplegar en Railway (recomendado)

Vas a crear **un proyecto con tres servicios**, los tres desde tu fork. Los nombres de los menús pueden variar un poco según la versión del panel de Railway.

12 Crea el servicio de la API

- En **railway.com**: **New Project** -> **Deploy from GitHub repo** -> elige tu fork **mlops2026** (la primera vez autoriza a Railway en GitHub).
- Railway detecta el `Dockerfile` y lee `railway.json` (health check en `/health`).
- Abre el servicio -> **Settings** -> cambia el nombre a **housing-api**. Los pasos siguientes usan ese nombre.

13 Agrega el token, el volumen y el dominio

Dónde	Qué hacer
Variables	**New Variable**: `ADMIN_TOKEN` = el token del paso 11
Volumen	Clic derecho sobre el servicio (o **Settings**) -> **Attach Volume** -> Mount path: `/data`
Settings -> Networking	**Generate Domain**. Obtendrás algo como https://housing-api-production.up.railway.app

> **Sin volumen se pierde todo**
>
> Sin el volumen en `/data`, cada redespliegue borra el modelo, los CSV subidos y el historial.

Espera a que el despliegue quede en **Active** y abre la URL: debe aparecer la interfaz con **API en servicio**.

14 Crea el cron de salud

- En el mismo proyecto: **New** (o **+ Create**) -> **GitHub Repo** -> otra vez tu fork.
- Nombre del servicio: **housing-health-check**.
- **Settings** -> archivo de configuración (*Config-as-code / Railway config file*): `deploy/railway.health.json`.

- Si no encuentras esa opción, configúralo a mano en **Settings**: **Custom Start Command** = `python cron_job.py health` y **Cron Schedule** = `*/15 * * * *`.

Variable	Valor (cópialo tal cual)
`API_URL`	`https://${{housing-api.RAILWAY_PUBLIC_DOMAIN}}`

`${{housing-api....}}` es una referencia de Railway: toma el dominio del servicio **housing-api**. No le agregues volumen ni dominio a este servicio.

15 Crea el cron de reentrenamiento

- **New** -> **GitHub Repo** -> tu fork. Nombre: **housing-retrain**.
- Archivo de configuración: `deploy/railway.retrain.json` (o a mano: Start Command `python cron_job.py retrain`, Cron Schedule `0 6 1,15 * *`).

Variable	Valor (cópialo tal cual)
`API_URL`	`https://${{housing-api.RAILWAY_PUBLIC_DOMAIN}}`
`ADMIN_TOKEN`	`${{housing-api.ADMIN_TOKEN}}`

16 Ejecuta cada cron a mano una vez

En cada cron usa **Run now** (o el botón equivalente) y abre sus **Logs**:

Servicio	Mensaje esperado
housing-health-check	`API sana · modelo v1 · predicción de prueba $...`
housing-retrain	`SE NECESITAN DATOS NUEVOS` (normal: aún no subiste datos)

Listo: pasa a la **Parte E** para verificar todo.

Parte D2. Desplegar en Render (alternativa)

> **Antes de empezar: costo**
>
> El modelo (RandomForest de 138 MB) usa unos 700 MB de RAM al reentrenar. Por eso `render.yaml` pide el plan **1c-2g** para la API; con 512 MB el reentrenamiento se queda sin memoria.
>
> El disco persistente y los cron jobs de Render **solo existen en planes de pago**. Revisa los precios vigentes en render.com/pricing antes de crear los servicios.

17 Crea los servicios con el Blueprint

- En **dashboard.render.com**: **New** -> **Blueprint** -> conecta GitHub y elige tu fork.
- Render lee `render.yaml` y propone tres servicios:

Servicio	Tipo	Qué hace
housing-api	Web service	La API, con disco de 1 GB montado en `/data`
housing-health-check	Cron job	`python cron_job.py health` cada 15 minutos
housing-retrain	Cron job	`python cron_job.py retrain` días 1 y 15, 06:00 UTC

18 Llena las variables que pide Render

Servicio	Variable	Valor
housing-api	`ADMIN_TOKEN`	El token del paso 11
housing-retrain	`ADMIN_TOKEN`	**El mismo** token
housing-health-check	`API_URL`	URL pública de la API
housing-retrain	`API_URL`	URL pública de la API

Si todavía no conoces la URL de la API, escribe cualquier valor y pulsa **Apply**. Cuando housing-api termine de desplegar, copia su URL (por ejemplo https://housing-api-xxxx.onrender.com) y cámbiala en **Environment** de cada cron.

19 Espera el despliegue y prueba los cron

- La API queda lista cuando su estado es **Live**; abre su URL para ver la interfaz.
- En cada cron pulsa **Trigger Run** y revisa los **Logs**: los mensajes esperados son los mismos de la tabla del paso 16.

Parte E. Verificación final

Reemplaza `<API>` por tu URL pública y marca cada punto:

[ ]	`<API>/health` responde `{"status":"healthy","model_loaded":true,"model_version":"v1"}`
[ ]	La interfaz (`<API>/`) muestra **API en servicio** y el campeón v1 con su RMSE.
[ ]	Una predicción desde el formulario devuelve un valor y aparece en **Monitoreo de predicciones**.
[ ]	Con tu token, **Reentrenar ahora** responde "No hay datos nuevos..." (no "Token de administración inválido").
[ ]	El log del cron de salud dice **API sana**.
[ ]	El log del cron de reentrenamiento dice **SE NECESITAN DATOS NUEVOS**.
[ ]	Subes un CSV de prueba desde la interfaz y aparece como **Pendiente**.
[ ]	Ejecutas el cron de reentrenamiento a mano y el historial muestra **Nuevo campeón** o **Se mantuvo el actual**.
[ ]	Redespliegas la API y el historial sigue ahí (el volumen funciona).

> **Cómo se actualiza**
>
> Cada `git push` a la rama main de tu fork redespliega los tres servicios automáticamente. El modelo y los datos del volumen se conservan.

Anexo 1. Formato del CSV de datos nuevos

Archivo con **encabezado**, separado por comas y con **punto decimal**. El orden de las columnas no importa y las columnas extra se ignoran.

Columna	Tipo	Regla
`longitude`	número	obligatorio
`latitude`	número	obligatorio
`housing_median_age`	número	>= 0
`total_rooms`	número	>= 0
`total_bedrooms`	número	>= 0, **puede ir vacío**
`population`	número	>= 0
`households`	número	>= 0
`median_income`	número	>= 0, en decenas de miles de USD (8.3 = 83.000 USD)
`ocean_proximity`	texto	`<1H OCEAN`, `INLAND`, `ISLAND`, `NEAR BAY` o `NEAR OCEAN`
`median_house_value`	número	**> 0**: el valor real de la vivienda en USD

Ejemplo (en el archivo, el encabezado va en una sola línea):

```
longitude,latitude,housing_median_age,total_rooms,total_bedrooms,population,households,
median_income,ocean_proximity,median_house_value
-122.23,37.88,41,880,129,322,126,8.3252,NEAR BAY,452600
-118.25,34.05,30,2000,,1200,450,3.1,<1H OCEAN,310000
```

- Mínimo **10 filas** y máximo **20 MB** por archivo.
- Si una fila tiene un error se rechaza el **archivo completo** y la interfaz indica cada problema con su número de línea (la línea 1 es el encabezado).
- Desde Excel en español: *Guardar como -> CSV (delimitado por comas)* y revisa que los decimales usen punto, no coma.
- También se puede subir desde la terminal:

```
curl -X POST <API>/data/upload -H "X-Admin-Token: <tu-token>" \
     -H "Content-Type: text/csv" --data-binary @nuevos.csv
```

Anexo 2. Solución de problemas

Síntoma	Causa probable	Solución
`uv: command not found`	uv no está en el PATH	Cierra y vuelve a abrir la terminal tras instalarlo
`Address already in use`	Ya hay algo en el puerto 8000	Apaga la otra API o usa `--port 8001`

Síntoma	Causa probable	Solución
/health responde 503 varios minutos	El modelo inicial sigue entrenando o falló	Revisa los logs de la API: busca "Campeón v1 listo" o "No se pudo preparar el modelo"
"Configure ADMIN_TOKEN..."	La API no tiene la variable	Agrega `ADMIN_TOKEN` a la API
"Token de administración inválido"	Token distinto o con espacios	Vuelve a pegarlo sin espacios; el del cron debe ser igual al de la API
Cron: "No se pudo contactar la API"	`API_URL` mal escrita o API caída	Debe empezar por https:// y no terminar en /
Cron: "Falta la variable de entorno"	Falta `API_URL` o `ADMIN_TOKEN`	Agrégala en las variables del cron
Tras redesplegar desaparece el historial	No hay volumen en /data	Monta el volumen en `/data`
La API se reinicia al reentrenar (Render)	Sin memoria (plan de 512 MB)	Plan 1c-2g o superior
CSV rechazado: "valores no numéricos"	Comas decimales o separador ;	Exporta con punto decimal y coma como separador
Siempre "Se mantuvo el actual"	Los datos nuevos no reducen el error	Normal con pocos datos; acumula más lotes
Docker: /data/upload responde 404	Imagen de la API desactualizada	`docker compose up -d --build`
Docker: puerto 8888 ocupado	Otro Jupyter usa ese puerto	`JUPYTER_PORT=8889` en `.env`
Advertencia "X does not have valid feature names"	Viene del pipeline exportado por PyCaret	Es inofensiva

Más detalle técnico en `MANUAL.md` y `README.md` del repositorio.